

Detailed Frontend API Integration Handoff

A screen-by-screen implementation guide covering every data panel, filter, button and workflow concern raised in the supplied PDF. It includes the exact endpoints, payloads, response shapes and frontend sequencing needed to deliver the reconciliation, checker review, posting authorization and verification experience.

Backend status: all endpoints required by the proposed screens are available.

-
- API base:** `{{base_url}}/api/cscs`
 - Authentication:** Sanctum bearer token
 - Postman:** `docs/postman/ProjectT-API.postman_collection.json`
 - Audience:** Project T frontend implementation team

1. What this answers from the annotated PDF

Concern raised	Answer
Which endpoint loads the reconciliation transaction table?	<code>GET /uploads/{batchId}/transactions?page=1&per_page=50</code>
Which endpoint loads the selected transaction card?	<code>GET /uploads/{batchId}/transactions/{transactionNumber}</code>
Which endpoint supplies batch totals/status cards?	<code>GET /uploads/{batchId}</code> ; optionally use <code>/preview</code> for a combined response.
Where does the exception detail panel get its data?	Use the selected row from <code>GET /uploads/{batchId}/exceptions</code> . Fetch transaction detail only when full debit/credit context is needed.
What payload confirms an account mapping?	Use the exception resolution endpoint with <code>resolution_type=MAP_ACCOUNT</code> , the selected <code>register_account_id</code> , and a reason.
What endpoints power replay, exclusion and cancellation?	Replay and exclusion use the same exception resolution endpoint. Cancellation uses <code>POST /uploads/{batchId}/cancel</code> .
How is the checker screen populated?	Combine preview, transactions, approvals, events and comments. These reads can run in parallel.
How are posting checks shown before authorization?	<code>GET /uploads/{batchId}/posting-readiness</code> returns a top-level readiness decision and every individual check.
How is the final verification screen populated?	<code>GET /uploads/{batchId}/verification-summary</code> provides one consolidated response.
Are the proposed PDF, CSV and Excel reports available?	Yes. They are all served through <code>GET /uploads/{batchId}/export</code> with <code>type</code> and <code>format</code> .

2. Global request and response rules

Headers

```
Accept: application/json
Authorization: Bearer <token>
Content-Type: application/json // JSON mutations only
```

For report downloads, request a blob/binary response. For the original file upload, use browser `FormData` and allow the browser to create the multipart boundary.

Three response envelopes

Kind	Shape
Single-record read	<code>{ "data": { ... } }</code>
Mutation	<code>{ "success": true, "message": "...", "data": { ... } }</code>
Paginated read	<code>{ "current_page": 1, "data": [], "last_page": 1, "per_page": 50, "total": 0, "meta": { ... } }</code>

Quantity handling: share quantities are returned as decimal strings such as `"248889.000000"` . Do not convert them to JavaScript floating-point numbers for equality or financial calculations. Display with the returned `meta.unit_precision` .

3. Screen 1 — Reconciliation workspace

Recommended initial load

```
const [batch, preview, transactions] = await Promise.all([
  api.get(`/api/cscs/uploads/${batchId}`),
  api.get(`/api/cscs/uploads/${batchId}/preview`),
  api.get(`/api/cscs/uploads/${batchId}/transactions`, {
    params: { page: 1, per_page: 50 }
  })
]);
```

Load exceptions immediately when the design displays an exception count, or lazily when the user continues to the exception workbench.

UI area	Source	Fields to use
Header/status	GET /uploads/{batchId}	workflow_status , business_reference , description , allowed_actions
Debit/credit cards	Batch summary or reconciliation	total_debit , total_credit , net_movement
Exception/transaction cards	Batch summary + transaction metadata	unresolved_exceptions , transaction_groups , meta.transaction_counts
Transaction table	GET /transactions	transaction_number , quantity , account objects, risk , resolution , balance and flags
Selected transaction	GET /transactions/{transactionNumber}	Same summary fields plus full legs
Account impact preview	GET /account-effects	Named shareholder/account/class plus current/debit/credit/net/proposed quantities
Security mapping	GET /preview	data.security_mappings

Transaction filters

```
GET /api/cscs/uploads/{batchId}/transactions
?search=2606160005615022
&balance_status=BALANCED
&is_flagged=false
&resolution_status=READY
&security_code=STANBIC
&trade_date_from=2026-06-16
&trade_date_to=2026-06-16
&page=1&per_page=50
```

All filters are optional and are applied before pagination. Use meta.transaction_counts for the All/Balanced/Unbalanced/Flagged tabs; do not calculate those totals from the current page.

Representative transaction object

```
{
  "transaction_number": "2606160005615022",
  "trade_date": "2026-06-16",
  "security_code": "STANBIC",
  "quantity": "248889.000000",
  "quantity_mismatch": false,
  "debit_total": "248889.000000",
  "credit_total": "248889.000000",
  "net_total": "0.000000",
  "leg_count": 2,
  "is_balanced": true,
  "balance_status": "BALANCED",
  "is_flagged": false,
  "flag_reasons": [],
  "risk": { "level": "LOW", "label": "Low", "reasons": [] },
  "resolution": { "status": "READY", "label": "Ready", "action_required": false },
  "debit_account": {
    "register_account_id": 445,
```

```

"shareholder_name": "Debit Holder",
"register_account_number": "SRA-00000445",
"chn": "C111111111",
"current_quantity": "300000.000000",
"proposed_quantity": "51111.000000"
},
"credit_account": { "register_account_id": 446, "shareholder_name": "Credit Holder" },
"status": ["READY"],
"legs": [ /* full staged movement rows */ ]
}

```

4. Screen 1 — Exception resolution workbench

List endpoint

```

GET /api/cscs/uploads/{batchId}/exceptions
?resolution_status=UNRESOLVED&search=C111111111&page=1&per_page=10

```

Both `status` and `resolution_status` are accepted. Search matches transaction, identifier, code, message or row number. Use `meta.exception_counts` for the Total, Blocking, Warning, Resolved and Remaining cards.

Detail panel behavior

1. Store the clicked exception row as `selectedException`.
2. Render `severity`, `is_blocking`, `parsed_record`, `suggested_resolution`, `allowed_resolution_types`, `matched_account` and `resolution_history` directly from that object.
3. If the design needs both movement legs, call `GET /transactions/{selectedException.tran_no}`.
4. After any resolution, clear the selection and refresh exceptions, batch, preview and transaction counts.

Confirm an existing account mapping

```

POST /api/cscs/uploads/{batchId}/exceptions/{exceptionId}/resolve

{
  "resolution_type": "MAP_ACCOUNT",
  "register_account_id": 445,
  "reason": "Confirmed against the shareholder register."
}

```

Required: `register_account_id` is the internal shareholder-register-account ID. Find it with `GET /api/shareholders?register_id={registerId}&search={query}`; search accepts names, shareholder numbers, CHNs and CSCS numbers.

Button	resolution_type	Additional rule
Confirm Mapping	MAP_ACCOUNT	<code>register_account_id</code> required; account must belong to the batch register.
Mark as Replay	CONFIRM_REPLAY	A matching previously posted movement must exist.
Exclude Record	RULE_EXCLUDED	Applies a documented exclusion to the complete transaction group.

```

// Replay
{ "resolution_type": "CONFIRM_REPLAY",
  "reason": "Confirmed as an already posted CSCS movement." }

// Exclusion
{ "resolution_type": "RULE_EXCLUDED",
  "reason": "Excluded under the approved documented CSCS rule." }

```

Every resolution reason is required and must contain 10–1000 characters. A successful response states that revalidation is required.

Creating a new shareholder/account from the panel

POST /api/shareholders → receive shareholder.id → POST /api/shareholders/{shareholderId}/register-accounts → receive registerAccount.id → POST CSCS exception resolve with MAP_ACCOUNT + registerAccount.id → POST /api/cscs/uploads/{batchId}/reconcile

Do not stop after account creation. The CSCS exception is unresolved until the final `MAP_ACCOUNT` call succeeds.

5. Completing the maker workflow

Button	Request	Frontend result
Revalidate / Reconcile	POST /uploads/{batchId}/reconcile { "comment": "All groups reviewed" }	Refresh the batch. Continue only when status becomes <code>RECONCILED</code> .
Submit for Approval	POST /uploads/{batchId}/submit	Status becomes <code>PENDING_APPROVAL</code> and a revision snapshot is frozen.
Cancel Batch	POST /uploads/{batchId}/cancel with a required 10+ character comment	Status becomes <code>CANCELLED</code> ; the batch is final/read-only.
Save & Continue Later	No API call required	All staged data and successful resolutions are already persisted. Navigate away after pending requests finish.

Submission gate

Before enabling submission, confirm all of the following from the refreshed batch:

- `workflow_status === "RECONCILED"`;
- `summary.unresolved_exceptions === 0`;
- debit and credit totals match;
- `snapshot_hash` is present; and
- `allowed_actions` contains `submit`.

Authoritative button list

```
{
  "data": {
    "id": 101,
    "workflow_status": "DRAFT_REVIEW",
    "allowed_actions": ["view", "resolve", "revalidate", "cancel"]
  }
}
```

Always combine `allowed_actions` with the frontend permission model. Refresh it after every mutation because workflow state and actor separation can change what is legal.

6. Workflow states the UI must understand

PROCESSING → DRAFT_REVIEW → RECONCILED → PENDING_APPROVAL ↗ QUERY_RAISED → DRAFT_REVIEW ↗ REJECTED ↘ APPROVED_AWAITING_POST → POSTING_QUEUED → POSTING ↗ POSTED ↘ POSTING_FAILED → STALE → DRAFT_REVIEW DRAFT_REVIEW / RECONCILED / QUERY_RAISED / STALE → CANCELLED

Status	Recommended UI
<code>PROCESSING</code>	Progress screen; poll batch details.
<code>DRAFT_REVIEW</code> , <code>QUERY_RAISED</code> , <code>STALE</code>	Maker workspace and exception tools.
<code>PENDING_APPROVAL</code>	Read-only checker review with approve/query/reject.
<code>APPROVED_AWAITING_POST</code>	Posting authorization for an eligible non-maker.
<code>POSTING_QUEUED</code> , <code>POSTING</code>	Disable mutations and poll posting status.

Status	Recommended UI
POSTED	Final verification and reports.

7. Screen 2 — Checker review

Load these reads in parallel

```
const [batch, preview, transactions, approvals, events, comments] =
  await Promise.all([
    api.get(`/api/cscs/uploads/${batchId}`),
    api.get(`/api/cscs/uploads/${batchId}/preview`),
    api.get(`/api/cscs/uploads/${batchId}/transactions`, { params: { per_page: 50 } }),
    api.get(`/api/cscs/uploads/${batchId}/approvals`),
    api.get(`/api/cscs/uploads/${batchId}/events`),
    api.get(`/api/cscs/uploads/${batchId}/comments`)
  ]);
```

Mockup section	Implementation
Batch Metadata & Origins	<code>preview.data.batch</code> , including files, timestamps, revision, register and submitter information.
Reconciliation Summary	<code>batch.reconciliation</code> or <code>GET /reconciliation</code> .
Proposed Holdings Preview	<code>preview.data.account_effects</code> .
Transaction Groups	Paginated <code>GET /transactions</code> .
Proposed New Accounts	<code>preview.data.proposed_new_accounts</code> , including the proposed master profile.
Security Mapping Checks	<code>preview.data.security_mappings</code> ; final posting checks come from posting readiness.
Risk & Exceptions Checks	<code>preview.data.review_summary.exceptions</code> and <code>checks</code> .
Approval Timeline	<code>preview.data.approval_timeline</code> or the standalone events/approvals reads.
Comments & Review Notes	<code>preview.data.comments</code> or paginated <code>GET /comments</code> .

Standalone comment

```
POST /api/cscs/uploads/{batchId}/comments
{ "comment": "Please verify the attached CSCS instruction reference." }
```

The response is HTTP `201` . Refresh the comments endpoint afterward; it is paginated and newest-first.

Checker buttons

```
// Approve
POST /api/cscs/uploads/{batchId}/approve
{ "comment": "Files, balances and holding effects verified." }

// Raise Query
POST /api/cscs/uploads/{batchId}/query
{ "comment": "Please confirm the selected account mapping.",
  "transaction_numbers": ["2606160005615022"],
  "row_ids": [501] }

// Reject
POST /api/cscs/uploads/{batchId}/reject
{ "comment": "Source movement details require correction." }
```

Approval is not posting. Approval changes the batch to `APPROVED_AWAITING_POST` after all required approval steps. It does not update live holdings and should not immediately show a “successfully posted” message.

8. Screen 3 — Posting authorization

Preflight endpoint for the modal

```
GET /api/cscs/uploads/{batchId}/posting-readiness
```

Representative response

```
{
  "data": {
    "batch_id": 101,
    "status": "APPROVED_AWAITING_POST",
    "snapshot_hash": "9e2f...",
    "ready": true,
    "checks": {
      "status_allows_posting": {
        "passed": true, "label": "Batch approved for posting"
      },
      "snapshot_hash_unchanged": {
        "passed": true, "label": "Snapshot hash unchanged"
      },
      "security_mappings_valid": {
        "passed": true, "label": "Security mappings active and unchanged"
      },
      "account_mappings_valid": {
        "passed": true, "label": "Account mappings remain valid"
      },
      "holdings_current": {
        "passed": true, "label": "Current holdings match the approved snapshot"
      },
      "movements_not_posted": {
        "passed": true, "label": "Movements have not already been posted"
      },
      "no_blocking_exceptions": {
        "passed": true, "label": "No blocking exceptions remain"
      }
    },
    "summary": {
      "records_to_post": 1250,
      "affected_accounts": 847,
      "blocking_exceptions": 0
    },
    "posting_policy": {
      "is_irreversible": true,
      "correction_method": "CONTROLLED_REVERSAL",
      "rollback_window_hours": null
    }
  }
}
```

Render each check using its supplied label. Disable “Authorize Posting” when any check fails, `data.ready` is false, or the batch response does not include `post` in `allowed_actions`.

Authorization request

```
POST /api/cscs/uploads/{batchId}/post
{ "comment": "Approved for controlled posting." }
```

A successful request returns HTTP `202 Accepted` with status `POSTING_QUEUED`. The user’s acknowledgement checkbox and operational comment are frontend fields; submit the comment after the checkbox has been checked.

Segregation of duties: the maker can never post their own batch. The active policy may also require the poster to be different from the checker. A 403 response is final authorization feedback, not a condition to bypass or retry automatically.

9. Posting monitor and final verification

Poll posting status

GET /api/cscs/uploads/{batchId}/posting-status

```
const active = new Set(['POSTING_QUEUED', 'POSTING']);

async function pollPosting(batchId, signal) {
  while (!signal.aborted) {
    const { data: response } = await api.get(
      `/api/cscs/uploads/${batchId}/posting-status`, { signal }
    );
    const posting = response.data;
    if (!active.has(posting.status)) return posting;
    await new Promise(resolve => setTimeout(resolve, 3000));
  }
}
```

Result	Frontend action
POSTED	Stop polling and load verification summary.
POSTING_FAILED	Show sanitized <code>failure_reason</code> . Offer retry only when <code>allowed_actions</code> permits posting.
STALE	Return the batch to the maker; holdings or snapshot changed and reconciliation/approval must repeat.

Cancel the polling request when the component unmounts. Never run automatic posting retries in a loop.

Screen 4 endpoint

GET /api/cscs/uploads/{batchId}/verification-summary

Representative response

```
{
  "data": {
    "batch_id": 101,
    "status": "POSTED",
    "verification_status": "VERIFIED",
    "posted_at": "2026-08-31T12:45:00Z",
    "metrics": {
      "records_posted": 1250,
      "transaction_groups_posted": 625,
      "failed_rows": 0,
      "duplicate_prevention_blocks": 0,
      "total_debit": "156400.000000",
      "total_credit": "156400.000000",
      "net_movement": "0.000000",
      "shareholder_records_updated": 847,
      "new_accounts_created": 3,
      "exceptions_resolved": 12,
      "replay_transactions": 0
    },
    "approved_totals": {
      "total_debit": "156400.000000",
      "total_credit": "156400.000000",
      "net_movement": "0.000000",
      "ready_rows": 1250
    },
    "comparison": [
      {
        "metric": "total_debit",
        "approved": "156400.000000",
        "actual": "156400.000000",
        "variance": "0.000000",
        "status": "MATCHED",
        "matched": true
      }
    ],
    "checks": {
      "posted_row_count": true,
      "share_transaction_count": true,
    }
  }
}
```

```
    "unique_replay_fingerprints": true,
    "debit_total": true,
    "credit_total": true,
    "net_movement": true,
    "holding_effects": true
  },
  "all_checks_passed": true,
  "verified_at": "2026-08-31T12:45:01Z"
}
```

Only render the green verified state when `verification_status === "VERIFIED"` and `all_checks_passed` is true. Render the reconciliation table directly from `comparison` ; approved, actual, variance and status are already supplied.

10. Screen 4 — Reports and downloads

Card in design	Request	Response
Audit Report	<code>GET /uploads/{batchId}/export?type=audit&format=pdf</code>	PDF
Posting Report	<code>GET /uploads/{batchId}/export?type=posting&format=csv</code>	CSV
Reconciliation Report	<code>GET /uploads/{batchId}/export?type=reconciliation&format=pdf</code>	PDF
Activity Log	<code>GET /uploads/{batchId}/export?type=activity&format=xls</code>	Excel XLS

`format=xlsx` is also supported for activity. CSV types also include `rows` , `exceptions` , `reconciliation` , `preview` , `posting` and `activity` .

Browser download helper

```
async function downloadCscsReport(batchId, type, format) {
  const response = await api.get(
    `/api/cscs/uploads/${batchId}/export`,
    { params: { type, format }, responseType: 'blob' }
  );

  const disposition = response.headers['content-disposition'] || '';
  const match = disposition.match(/filename="?([^\"]+)"?/i);
  const filename = match?.[1] || `cscs_${batchId}_${type}.${format}`;
  const url = URL.createObjectURL(response.data);
  const link = document.createElement('a');
  link.href = url;
  link.download = filename;
  link.click();
  URL.revokeObjectURL(url);
}
```

Authorization: report downloads require `cscs.export` . Do not attempt to open the URL in an unauthenticated browser tab if authentication is provided only through a bearer header.

11. Permissions by action

Capability	Permission
Read batch, transactions, preview, history, comments and verification	<code>cscs.view</code>
Resolve exceptions and reconcile	<code>cscs.reconcile</code>
Submit or cancel maker-owned batch	<code>cscs.submit</code>
Raise checker query	<code>cscs.review</code>
Approve or reject	<code>cscs.approve</code>
Authorize posting	<code>cscs.post</code>

Capability	Permission
Download reports/source files	<code>cscs.export</code>

12. Error handling contract

Status	Meaning	Required UI behavior
401	Token missing or expired	End the session or refresh authentication.
403	Permission or maker-checker separation failure	Disable the action and display the API message.
404	Batch, row, account, transaction or file not found	Clear stale selection or return to the batch list.
422	Validation/state/reconciliation/readiness failure	Show validation errors and refresh current batch state.
429	Rate limit exceeded	Prevent repeated clicks and retry after a delay.
500	Sanitized technical failure	Show the safe message/reference; do not expose raw stack data.

Laravel validation example

```
{
  "message": "The reason field is required.",
  "errors": {
    "reason": ["The reason field is required."]
  }
}
```

Use a shared error helper that checks `response.data.errors`, then `response.data.message`, and finally a generic fallback.

13. Implementation checklist

1. Use the numeric batch ID returned by the upload/import response.
2. Load batch, preview and transactions in parallel.
3. Use server pagination and `meta.transaction_counts`.
4. Use `meta.exception_counts` for exception cards and tabs.
5. Use decimal strings and register precision for quantities.
6. Render buttons from `allowed_actions` plus frontend permissions.
7. Refresh relevant reads after every successful mutation.
8. Complete shareholder creation → account creation → CSCS mapping as one UI sequence.
9. Reconcile before allowing submission.
10. Keep approval and posting as distinct steps and messages.
11. Call posting readiness immediately before displaying/confirming posting.
12. Poll posting status only while queued or posting; cancel on unmount.
13. Show verified success only from verification summary.
14. Download reports as authenticated blobs.
15. Test 403, 404, 422, 429 and stale-snapshot behavior.

14. Postman and supporting references

- `docs/postman/ProjectT-API.postman_collection.json` — importable collection containing every registered API route.
- Set collection variables `base_url`, `token`, `batchId`, `exceptionId` and `transactionNumber`.
- For report requests, set `cscs_export_type` and `cscs_export_format`.
- `docs/CSCS_FRONTEND_INTEGRATION_GUIDE.md` — full operational frontend sequence.
- `docs/CSCS_API.md` — backend contract and workflow controls.

Final answer to the annotated request: the enriched API now supplies UI-ready transaction accounts/risk/resolution, exception severity/counts/suggestions/history, named review data, posting policy, and final approved-versus-actual comparison rows. No panel in the four supplied screens requires mock-only data.